МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Наследование

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2024 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ, с использованием механизма наследования.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.

2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы

(при нехватке заданий вариант задания вычисляется как номер_в_списке_группы - количество_заданий).

3. Разработать классы согласно варианту задания.

4. Написать и отладить программу на подготовленных наборах тестовых данных.

5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:

• титульный лист;

• цель лабораторной работы;

• постановку задачи;

• схему наследования классов (UML диаграмма классов);

• текст программы с комментариями;

• демонстрацию работы программы (Снимки экрана при выполнении действий

программы с описанием).

• краткие ответы на контрольные вопросы;

• выводы по лабораторной работе.

6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу, в которой будет реализовано несколько классов (согласно варианту задания).

В реализуемой программе необходимо предусмотреть возможность демонстрации работы конструкторов, деструкторов и методов созданных классов.

В зависимости от оценки, на которую вы претендуете, необходимо выполнить следующие задания (Для каждой следующей оценки нужно выполнить ВСЕ предыдущие задания, если обратное не указано явно):

Удовлетворительно

1. Определить базовый класс (обозначим его Ab) с обычными и чистыми виртуальными методами (согласно варианту задания).
Все поля должны быть защищёнными (protected).
Все методы, кроме статических (static), работают только с одним или двумя объектами класса (не массивом).
2. Определить классы-наследники класса Ab (согласно варианту задания).
Реализовать в них чистые виртуальные методы класса Ab.
Все поля должны быть закрытыми (private).
Все методы работают только с одним или двумя объектами (не массивом).
3. Реализовать конструкторы классов: - по умолчанию; - копирования (получающий параметр «ссылка на класс»)
Реализовать виртуальные деструкторы классов.
4. В каждом классе реализовать виртуальный метод myName(), который возвращает строку с названием класса.
5. В каждом классе написать методы ввода-вывода данных.
6. В функции main создать массив указателей на абстрактный класс Ab и заполнить его адресами объектов всех классов наследников. Произвести заполнение объектов и их вывод на экран.

Хорошо

7. Написать функции или статические методы базового класса Ab для вывода списков, предусмотренных вариантом.
8. Реализовать перегрузку операций >> и << с использованием виртуальных методов для ввода и вывода значений полей.
9. В функции main заполнять массив указателей (из задания 6) с помощью текстового файла. Выводить данные также в файл.

Отлично

10. Реализовать класс (обозначим его Ms), полями которого будет массив указателей на абстрактный класс Ab и количество элементов в этом массиве.

Вывод списков из задания 7 сделать отдельными методами класса Ms (в этом случае можно писать их сразу в классе Ms, а не отдельными функциями).

11. В функции main создать объект класса Ms и работать с ним.

12. С помощью функций реализовать меню для организации взаимодействия с пользователем.

С помощью меню необходимо выполнить демонстрацию функционирования всей программы (т.е. после выполнения действия происходит вывод меню до тех пор, пока в меню не будет выбран пункт «Закончить работу с программой»).

Вариант №5.

Создать класс **Payment**, включающий дату и сумму платежа.

На его основе реализовать классы:

- **BankTransfer** (Банковский перевод) с информацией о банке;
- **WebMoney** (Служба WebMoney) с информацией о размере комиссии.

Для массива указателей на объекты этих классов предусмотреть возможность: - вывода списка платежей на заданную дату; - вывода списка платежей, у которых величина платежа лежит в заданном диапазоне; - вывода статистической сводки по каждому типу платежа (сколько было платежей, их доля, общая денежная сумма для платежей данного типа).

Создать класс **Sales**, содержащий массив указателей на объекты этих классов.

Написать демонстрационную программу, в которой будут использоваться все методы классов.

ХОД РАБОТЫ

Файл **payment.h**:

Файл **payment.cpp**:

```cpp
#ifndef PAYMENT_H
#define PAYMENT_H

#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Payment {
protected:
    string date;
    float amount;

public:
    Payment();
    Payment(const string& date, float amount);
    Payment(const Payment& other);
    virtual ~Payment();

    virtual Payment* getCopy() const = 0;

    virtual string myName() const = 0;

    virtual void input(istream& is);
    virtual void output(ostream& os) const;

    // Геттеры
    string getDate() const { return date; }
    float getAmount() const { return amount; }

    // Виртуальные операторы ввода/вывода
    friend istream& operator>>(istream& is, Payment& p);
    friend ostream& operator<<(ostream& os, const Payment& p);
};

class BankTransfer : public Payment {
private:
    string bank;

public:
    BankTransfer();
    BankTransfer(const string& date, float amount, const string& bank);
    BankTransfer(const BankTransfer& other);
    virtual ~BankTransfer();

    virtual Payment* getCopy() const override;

    virtual string myName() const override;

    virtual void input(istream& is) override;
    virtual void output(ostream& os) const override;

    string getBank() const { return bank; }
};

class WebMoney : public Payment {
private:
    float commission;

public:
```

```cpp
class WebMoney : public Payment {
private:
    float commission;

public:
    WebMoney();
    WebMoney(const string& date, float amount, float commission);
    WebMoney(const WebMoney& other);
    virtual ~WebMoney();

    virtual Payment* getCopy() const override;

    virtual string myName() const override;

    virtual void input(istream& is) override;
    virtual void output(ostream& os) const override;

    float getCommission() const { return commission; }
};

class Sales {
private:
    vector<Payment*> payments;

public:
    Sales();
    Sales(const Sales& other);
    ~Sales();

    void addPayment(Payment* p);
    void displayAll() const;

    void selectByDate(const string& date) const;
    void selectByAmountRange(float min, float max) const;
    void showStatsByType() const;

    void loadFromFile(const string& filename);
    void saveToFile(const string& filename) const;
};

#endif
```

```cpp
#include "payment.h"
#include <fstream>
#include <algorithm>
#include <iomanip>

// Конструкторы

Payment::Payment() : date(""), amount(0.0f) {}
Payment::Payment(const string& date, float amount) : date(date), amount(amount) {}
Payment::Payment(const Payment& other) : date(other.date), amount(other.amount) {}
Payment::~Payment() {}

/// @brief Заполнение полей объекта
void Payment::input(istream& is) {
    if (&is == &cin) {
        cout << "Enter date: ";
    }
    is >> date;

    if (&is == &cin) {
        cout << "Enter amount: ";
    }
    is >> amount;
}

/// @brief Вывод полей объекта
void Payment::output(ostream& os) const {
    os << "Date: " << date << ", Amount: " << amount;
}

/// @brief Надстройка над input()
/// @param is Входящий поток
/// @param p Объект для ввода
/// @return Изменённый поток
istream& operator>>(istream& is, Payment& p) {
    p.input(is);
    return is;
}

/// @brief Надстройка над output()
/// @param os Исходящий поток
/// @param p Объект для вывода
/// @return Изменённый поток
ostream& operator<<(ostream& os, const Payment& p) {
    p.output(os);
    return os;
}



// Конструкторы

BankTransfer::BankTransfer() : Payment(), bank("") {}
BankTransfer::BankTransfer(const string& date, float amount, const string& bank)
    : Payment(date, amount), bank(bank) {}
BankTransfer::BankTransfer(const BankTransfer& other)
    : Payment(other), bank(other.bank) {}
BankTransfer::~BankTransfer() {}

Payment* BankTransfer::getCopy() const {
```

```
Payment* BankTransfer::getCopy() const {
    return new BankTransfer(*this);
}

/// @brief Геттер для названия объекта
/// @return Строка с названием
string BankTransfer::myName() const {
    return "BankTransfer";
}

/// @brief Заполнение полей объекта
void BankTransfer::input(istream& is) {
    Payment::input(is);
    if (&is == &cin) {
        cout << "Enter bank: ";
    }
    is >> bank;
}

/// @brief Вывод полей объекта
void BankTransfer::output(ostream& os) const {
    Payment::output(os);
    os << ", Bank: " << bank;
}

/// @brief Надстройка над input()
/// @param is Входящий поток
/// @param p Объект для ввода
/// @return Изменённый поток
istream& operator>>(istream& is, BankTransfer& b) {
    b.input(is);
    return is;
}

/// @brief Надстройка над output()
/// @param os Исходящий поток
/// @param p Объект для вывода
/// @return Изменённый поток
ostream& operator<<(ostream& os, const BankTransfer& b) {
    b.output(os);
    return os;
}


// Конструкторы

WebMoney::WebMoney() : Payment(), commission(0.0f) {}
WebMoney::WebMoney(const string& date, float amount, float commission)
    : Payment(date, amount), commission(commission) {}
WebMoney::WebMoney(const WebMoney& other)
    : Payment(other), commission(other.commission) {}
WebMoney::~WebMoney() {}

Payment* WebMoney::getCopy() const {
    return new WebMoney(*this);
}

/// @brief Геттер для названия объекта
/// @return Строка с названием
string WebMoney::myName() const {
```

```
/// @brief Геттер для названия объекта
/// @return Строка с названием
string WebMoney::myName() const {
    return "WebMoney";
}

/// @brief Заполнение полей объекта
void WebMoney::input(istream& is) {
    Payment::input(is);
    if (&is == &cin) {
        cout << "Enter commission: ";
    }
    is >> commission;
}

/// @brief Вывод полей объекта
void WebMoney::output(ostream& os) const {
    Payment::output(os);
    os << ", Commission: " << commission;
}

/// @brief Надстройка над input()
/// @param is Входящий поток
/// @param p Объект для ввода
/// @return Изменённый поток
istream& operator>>(istream& is, WebMoney& w) {
    w.input(is);
    return is;
}

/// @brief Надстройка над output()
/// @param os Исходящий поток
/// @param p Объект для вывода
/// @return Изменённый поток
ostream& operator<<(ostream& os, const WebMoney& w) {
    w.output(os);
    return os;
}


// Конструкторы

Sales::Sales() {}
Sales::Sales(const Sales& other) {
    for (const auto& p : other.payments) {
        addPayment(p->getCopy());
    }
}
Sales::~Sales() {
    for (auto p : payments) {
        delete p;
    }
}

/// @brief Добавление объекта в массив
/// @param p Поинтер на массив
void Sales::addPayment(Payment* p) {
    payments.push_back(p);
}
```

```cpp
/// @brief Добавление объекта в массив
/// @param p Поинтер на массив
void Sales::addPayment(Payment* p) {
    payments.push_back(p);
}

/// @brief Вывод всех объектов массива
void Sales::displayAll() const {
    for (auto p : payments) {
        cout << p->myName() << ": ";
        cout << p << endl;
    }
}

/// @brief Вывод объектов из массива на заданную дату
/// @param date Заданная дата
void Sales::selectByDate(const string& date) const {
    cout << "\n=== Payments on " << date << " ===" << endl;
    bool found = false;
    for (auto p : payments) {
        if (p->getDate() == date) {
            cout << p->myName() << ": ";
            cout << p << endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No payments found on this date." << endl;
    }
}

/// @brief Вывод объектов из массива в диапазоне величины платежа (включительно)
/// @param min Минимальная допустимая величина
/// @param max Максимальная допустимая величина
void Sales::selectByAmountRange(float min, float max) const {
    cout << "\n=== Payments between " << min << " and " << max << " ===" << endl;
    bool found = false;
    for (auto p : payments) {
        if (p->getAmount() >= min && p->getAmount() <= max) {
            cout << p->myName() << ": ";
            cout << p << endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No payments found in this range." << endl;
    }
}

/// @brief Статистическая сводка по всем типам платежа.
/// Количество операций, общая сумма по типу, процент от суммы операций всех типов
void Sales::showStatsByType() const {
    int bankCount = 0, webCount = 0;
    float bankSum = 0, webSum = 0;

    for (auto p : payments) {
        if (p->myName() == "BankTransfer") {
            bankCount++;
            bankSum += p->getAmount();
```

```cpp
/// @brief Статистическая сводка по всем типам платежа.
/// Количество операцией, общая сумма по типу, процент от суммы операций всех типов
void Sales::showStatsByType() const {
    int bankCount = 0, webCount = 0;
    float bankSum = 0, webSum = 0;

    for (auto p : payments) {
        if (p->myName() == "BankTransfer") {
            bankCount++;
            bankSum += p->getAmount();
        } else if (p->myName() == "WebMoney") {
            webCount++;
            webSum += p->getAmount();
        }
    }

    float total = bankSum + webSum;

    cout << "\n=== Statistics by Type ===" << endl;
    cout << "BankTransfer: " << bankCount << " payments, sum = " << bankSum
         << ", share = " << fixed << setprecision(2) << (total ? (bankSum / total) * 100 : 0) << "%" << endl;
    cout << "WebMoney: " << webCount << " payments, sum = " << webSum
         << ", share = " << fixed << setprecision(2) << (total ? (webSum / total) * 100 : 0) << "%" << endl;
}

/// @brief Загрузка данных из файла
/// @param filename Файл с данными
void Sales::loadFromFile(const string& filename) {
    ifstream file(filename);

    if (!file.is_open()) {
        cout << "Error: Cannot open file " << filename << endl;
        return;
    }

    payments.clear();

    string type;
    while (file >> type) {
        Payment* obj;
        if (type == "BankTransfer") {
            obj = new BankTransfer;
            file >> *obj;
            addPayment(obj);
        }
        if (type == "WebMoney") {
            obj = new WebMoney;
            file >> *obj;
            addPayment(obj);
        }
    }

    file.close();
    cout << "Data loaded from " << filename << endl;
}

/// @brief Запись данных в файл
/// @param filename Файл для записи
void Sales::saveToFile(const string& filename) const {
    ofstream file(filename);
```

```
/// @brief Запись данных в файл
/// @param filename Файл для записи
void Sales::saveToFile(const string& filename) const {
    ofstream file(filename);

    if (!file.is_open()) {
        cout << "Error: Cannot open file " << filename << endl;
        return;
    }

    for (auto p : payments) {
        file << p->myName() << " " << p->getDate() << " " << p->getAmount() << " ";
        if (auto* bt = dynamic_cast<BankTransfer*>(p)) {
            file << bt->getBank();
        }
        if (auto* wm = dynamic_cast<WebMoney*>(p)) {
            file << wm->getCommission();
        }
        file << endl;
    }

    file.close();
    cout << "Data saved to " << filename << endl;
}
```

Файл **main.cpp**:

```cpp
#include "payment.h"
#include "payment.cpp"
#include <iostream>
using namespace std;

const string INPUT_FILENAME = "input.txt";
const string OUTPUT_FILENAME = "output.txt";


int main() {
    Sales sales;
    bool running = true;

    while (running) {
        cout << "\n====================================" << endl;
        cout << "            PAYMENT SYSTEM MENU" << endl;
        cout << "====================================" << endl;
        cout << "1. Add Bank Transfer" << endl;
        cout << "2. Add WebMoney" << endl;
        cout << "3. Display All Payments" << endl;
        cout << "4. Select by Date" << endl;
        cout << "5. Select by Amount Range" << endl;
        cout << "6. Show Statistics by Type" << endl;
        cout << "7. Load from File" << endl;
        cout << "8. Save to File" << endl;
        cout << "9. Exit" << endl;
        cout << "Choose an action: ";

        int choice;
        cin >> choice;

        switch (choice) {
            case 1: {
                string date, bank;
                float amount;
                cout << "Enter date: ";
                cin >> date;
                cout << "Enter amount: ";
                cin >> amount;
                cout << "Enter bank: ";
                cin >> bank;
                sales.addPayment(new BankTransfer(date, amount, bank));
                break;
            }
            case 2: {
                string date;
                float amount, commission;
                cout << "Enter date: ";
                cin >> date;
                cout << "Enter amount: ";
                cin >> amount;
                cout << "Enter commission: ";
                cin >> commission;
                sales.addPayment(new WebMoney(date, amount, commission));
                break;
            }
            case 3:
                sales.displayAll();
                break;
            case 4: {
                string date;
                cout << "Enter date: ";
                cin >> date;
                sales.selectByDate(date);
                break;
            }
            case 5: {
```

```
            case 5: {
                float min, max;
                cout << "Enter min amount: ";
                cin >> min;
                cout << "Enter max amount: ";
                cin >> max;
                sales.selectByAmountRange(min, max);
                break;
            }
            case 6:
                sales.showStatsByType();
                break;
            case 7: {
                sales.loadFromFile(INPUT_FILENAME);
                break;
            }
            case 8: {
                sales.saveToFile(OUTPUT_FILENAME);
                break;
            }
            case 9:
                running = false;
                break;
            default:
                cout << "Invalid choice. Try again." << endl;
        }
    }

    return 0;
}
```

Загрузка данных из файла:

```
=====================================
            PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 7
Data loaded from input.txt

=====================================
            PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 3
BankTransfer: Date: 01.01, Amount: 500, Bank: BANK_1
WebMoney: Date: 01.04, Amount: 750, Commission: 5
BankTransfer: Date: 05.10, Amount: 13, Bank: BANK_2
WebMoney: Date: 20.04, Amount: 1002, Commission: 10
BankTransfer: Date: 31.12, Amount: 10000, Bank: BANK_3
WebMoney: Date: 26.08, Amount: 4200, Commission: 13
```

Добавление новых записей о переводах:

```
=====================================
            PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 1
Enter date: 12.10
Enter amount: 2000
Enter bank: SOME_BANK

=====================================
            PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 2
Enter date: 12.10
Enter amount: 2500
Enter commission: 10

=====================================
            PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 3
BankTransfer: Date: 01.01, Amount: 500, Bank: BANK_1
WebMoney: Date: 01.04, Amount: 750, Commission: 5
BankTransfer: Date: 05.10, Amount: 13, Bank: BANK_2
WebMoney: Date: 20.04, Amount: 1002, Commission: 10
BankTransfer: Date: 31.12, Amount: 10000, Bank: BANK_3
WebMoney: Date: 26.08, Amount: 4200, Commission: 13
BankTransfer: Date: 12.10, Amount: 2000, Bank: SOME_BANK
WebMoney: Date: 12.10, Amount: 2500, Commission: 10
```

Вывод записей от определённой даты:

```
=====================================
           PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 4
Enter date: 12.10

=== Payments on 12.10 ===
BankTransfer: Date: 12.10, Amount: 2000, Bank: SOME_BANK
WebMoney: Date: 12.10, Amount: 2500, Commission: 10
```

Вывод записей с величиной платежа в определённом диапазоне:

```
=====================================
           PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 5
Enter min amount: 1000
Enter max amount: 5000

=== Payments between 1000 and 5000 ===
WebMoney: Date: 20.04, Amount: 1002, Commission: 10
WebMoney: Date: 26.08, Amount: 4200, Commission: 13
BankTransfer: Date: 12.10, Amount: 2000, Bank: SOME_BANK
WebMoney: Date: 12.10, Amount: 2500, Commission: 10
```

Вывод статистики по всем платёжным операциям:

```
=====================================
           PAYMENT SYSTEM MENU
=====================================
1. Add Bank Transfer
2. Add WebMoney
3. Display All Payments
4. Select by Date
5. Select by Amount Range
6. Show Statistics by Type
7. Load from File
8. Save to File
9. Exit
Choose an action: 6

=== Statistics by Type ===
BankTransfer: 4 payments, sum = 12513, share = 59.69%
WebMoney: 4 payments, sum = 8452.00, share = 40.31%
```

Запись данных массива в файл:

output.txt U ×

C_codespace > 2 kurs > 3-rd senester > Programming la

```
BankTransfer 01.01 500 BANK_1
WebMoney 01.04 750 5
BankTransfer 05.10 13 BANK_2
WebMoney 20.04 1002 10
BankTransfer 31.12 10000 BANK_3
WebMoney 26.08 4200 13
BankTransfer 12.10 2000 SOME_BANK
WebMoney 12.10 2500 10
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чем состоит сущность механизма наследования как принципа ООП?

Механизм наследования позволяет создавать новые классы на основе уже существующих, при этом производный класс автоматически получает все поля, методы и свойства базового класса (если они доступны).

2. Как объявляется производный класс?

class Derived : public Base {тело класса};

3. Как осуществляется доступ к элементам базового класса из производного при наследовании?

Доступ зависит от типа наследования:

- Public наследование:
  - public члены остаются public
  - protected члены остаются protected
  - private члены недоступны напрямую, но могут быть доступны через public/protected методы базового класса
- Protected наследование:
  - public и protected становятся protected
  - private недоступны
- Private наследование:
  - Все члены становятся private

4. Что производный класс наследует из базового класса?

- Элементы всех полей, кроме private.
- Конструкторы, но неявно – их необходимо определять вручную.

5. Что производный класс не наследует из базового класса?

- Элементы private поля.
- Операторы присваивания
- Дружественные функции
- Статические члены

6. Каким образом выполняется инициализация наследуемых элементов?

Через вызов конструктора базового класса в списке инициализации конструктора производного класса.

7. Что такое множественное наследование? Как выполняются конструкторы базовых классов при множественном наследовании?

Множественное наследование — это возможность производного класса наследовать сразу от нескольких базовых классов. Конструкторы базовых классов вызываются в порядке объявления базовых классов в списке наследования.

8. В каком порядке вызываются конструкторы и деструкторы класса при создании и удалении объекта производного класса?

- При создании объекта:

    - Сначала вызываются конструкторы базовых классов в порядке их объявления (слева направо).
    - Затем — конструктор производного класса.
- При удалении объекта:
    - Сначала вызывается деструктор производного класса.
    - Затем — деструкторы базовых классов в обратном порядке объявления.

9. Какой принцип ООП реализует механизм виртуальных функций?

Механизм виртуальных функций реализует принцип полиморфизма — способность объектов разных классов (связанных наследованием) обрабатывать одинаковые сообщения (вызовы методов) по-разному.

10. Когда следует использовать механизм виртуальных функций в программе?

- Когда нужно динамическое связывание: вызов нужной версии метода в зависимости от типа объекта во время выполнения.
- Когда есть иерархия классов, где производные классы должны переопределять поведение базового класса.
- Когда требуется использовать полиморфизм — работать с объектами разных классов через единый интерфейс (базовый класс).

11. В чем состоит сущность метода динамического связывания?

Динамическое связывание — это метод, при котором выбор конкретной версии метода класса, в зависимости от типа объекта, происходит во время выполнения программы, а не на этапе компиляции.

12. В чем смысл идеи «один интерфейс – множество методов»?

Это основная идея полиморфизма: один и тот же интерфейс может иметь разные реализации в разных классах.

13. Когда может быть проигнорирован механизм виртуальных функций?

- Вызов метода происходит через объект, а не через указатель/ссылку на базовый класс.
- Метод не является виртуальным — тогда вызов жёстко привязан к типу переменной.
- Используется прямой вызов метода базового класса через Base::method(), что обходит полиморфизм.

14. Что такое абстрактный класс? Можно ли создавать объекты абстрактных классов?

Абстрактный класс — это класс, который содержит хотя бы одну чисто виртуальную функцию (в C++ — virtual void f() = 0;). Такой класс не может быть инстанцирован напрямую — нельзя создать объект абстрактного класса.

ВЫВОДЫ

Я изучил принципы механизмов наследования и полиморфизма и научился применять их при разработке программ на языке C++.